# Markdown Reader

Markdown Reader is a clean and intuitive Markdown reader with real-time preview support, dark mode toggle, and drag-and-drop file opening. It is compatible with macOS and Windows desktop environments and built with pure Python and Tkinter.

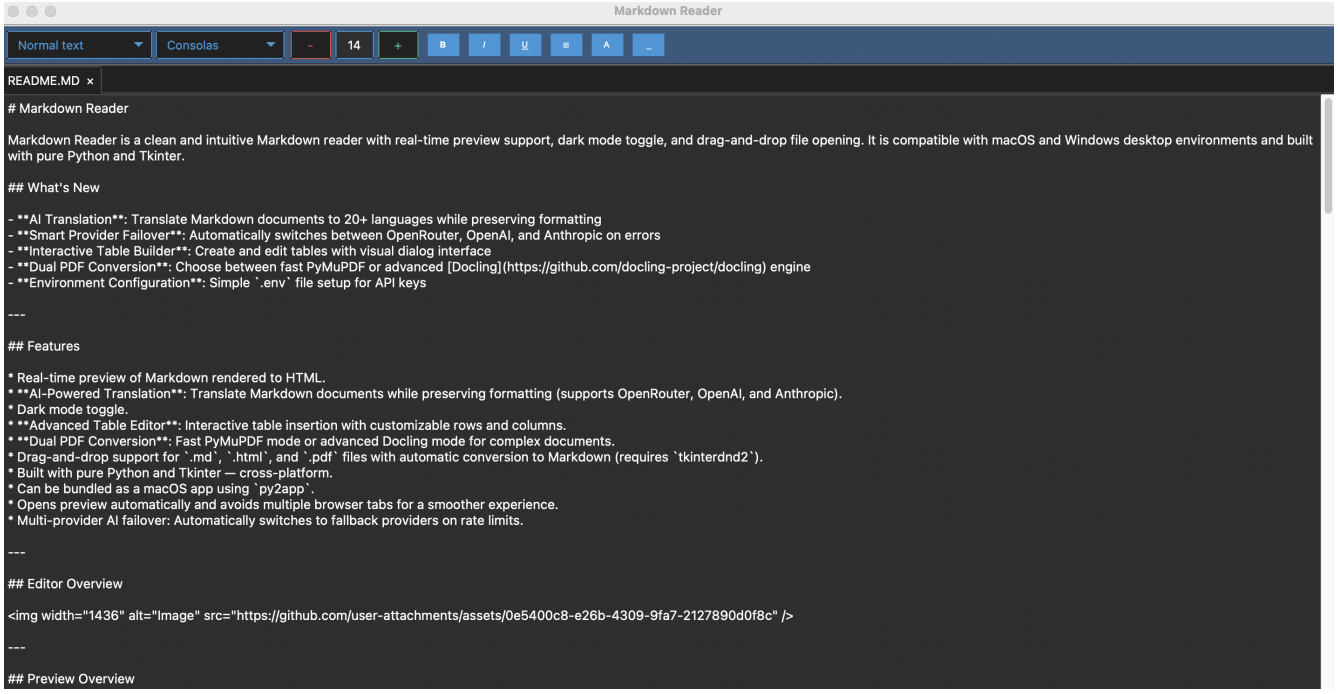
---

## Features

- Real-time preview of Markdown rendered to HTML.
- **AI-Powered Translation**: Translate Markdown documents while preserving formatting (supports OpenRouter, OpenAI, and Anthropic).
- Dark mode toggle.
- **Advanced Table Editor**: Interactive table insertion with customizable rows and columns.
- **Dual PDF Conversion**: Fast PyMuPDF mode or advanced Docling mode for complex documents.
- Drag-and-drop support for `.md`, `.html`, and `.pdf` files with automatic conversion to Markdown.
- Built with pure Python and Tkinter — cross-platform.
- Can be bundled as a macOS app using `py2app`.
- Opens preview automatically and avoids multiple browser tabs for a smoother experience.
- Multi-provider AI failover: Automatically switches to fallback providers on rate limits.

---

# Editor Overview




---

# Preview Overview

## Installation & Usage

### 1. Clone the repository

```
Copy
git clone https://github.com/petertzy/markdown-reader.git
cd markdown-reader
```

### 2. Create a virtual environment (recommended)

```
Copy
python -m venv venv
source venv/bin/activate  # macOS/Linux
# .\venv\Scripts\activate  # Windows (cmd/powershell)
```

---

# Installation & Usage

## 1. Clone the repository

```
git clone https://github.com/petertzy/markdown-reader.git
cd markdown-reader
```

### 2. Create a virtual environment (recommended)

```
python -m venv venv
source venv/bin/activate  # macOS/Linux
# .\venv\Scripts\activate  # Windows (cmd/powershell)
```

### 3. Install dependencies

For Mac users, you should first complete the preparation steps described in the PrepareForMacUser file.

For Windows users, WeasyPrint requires additional system libraries. Complete the preparation steps described in the PrepareForWindowsUser file before running `pip install`.

```
pip install -r requirements.txt
```

---

# Running the Application

```
python app.py
```

## How to Use

- **Open File**: Choose `.md`, `.html`, or `.pdf` file from the "File → Open File" menu.
- **Drag-and-Drop**: Drag `.md`, `.html`, or `.pdf` files directly into the app window (HTML and PDF files are automatically converted to Markdown).
- **Dark Mode**: Toggle via "View → Toggle Dark Mode".
- **Preview**: Automatically opens in your web browser, only one tab is opened per session.
- **AI Translation**:
- Translate selected text: "Edit → Translate with AI → Translate Selected Text with AI"
- Translate full document: "Edit → Translate with AI → Translate Full Document with AI"
- Select source and target languages from dropdown menus
- Switch AI provider: "Settings → AI Provider"
- **Table Insertion**: Use "Table → Insert Table" to create custom tables with interactive cell editing.
- **PDF Conversion Modes**:
- Fast mode (default): Uses PyMuPDF for quick extraction

- Advanced mode: Enable "Tools → Use Advanced PDF Conversion (Docling)" for complex documents
- View mode info: "Tools → PDF Converter Info"

---

## Packaging as a macOS App (Optional)

To bundle this app as a `.app`, install `py2app` and create a `setup.py` script:

### Build the App

```
rm -rf build dist
python setup.py py2app
```

The generated app will be located in the `dist/` folder. You can launch it by double-clicking. To use it like a regular app, move it to your Applications folder.

---

## Exit the Virtual Environment

```
deactivate
```

## Submit Changes to Git

```
git add .
git commit -m "Update"  # Replace "Update" with a meaningful commit message
git push
```

## Technical Details

- GUI: `tkinter`, `ttkbootstrap`
- Markdown Engine: `markdown2`
- HTML Preview: Dynamically generated HTML opened in the default browser
- File Conversion: `html2text` for HTML, `pypdf` and `docling` for PDF import
- PDF Export: Browser-based print-to-PDF (no system dependencies required)
- AI Translation: `requests` for API communication with OpenRouter, OpenAI, and Anthropic

- Configuration: Environment variables via `.env` file
- Supports drag-and-drop and live preview updates while avoiding multiple browser tabs
- Auto-failover: Automatically switches AI providers on rate limits or errors

---

## System Requirements:

Python >= 3.10

---

## AI-powered translation:

To enable AI-powered translation features, you need to set up API keys:

```
# Copy the example configuration file
cp .env.example .env
```

**How to get API keys:**
- **OpenRouter** (recommended for free tier): openrouter.ai
- **OpenAI**: platform.openai.com
- **Anthropic**: console.anthropic.com
**Solutions**:

The app will automatically switch to a fallback provider if the primary provider hits rate limits.

---

## License

This project is licensed under the **MIT License**.
See the LICENSE file for the full text.

---

## Contributing

All contributions are welcome, including:

- Bug reports
- Feature suggestions
- Pull requests
- Documentation improvements

Please see our CONTRIBUTING guide for more details on how to get started, submit changes, or report issues.